

Discrete Flow Matching Policy: Learning to Act in Discrete Space

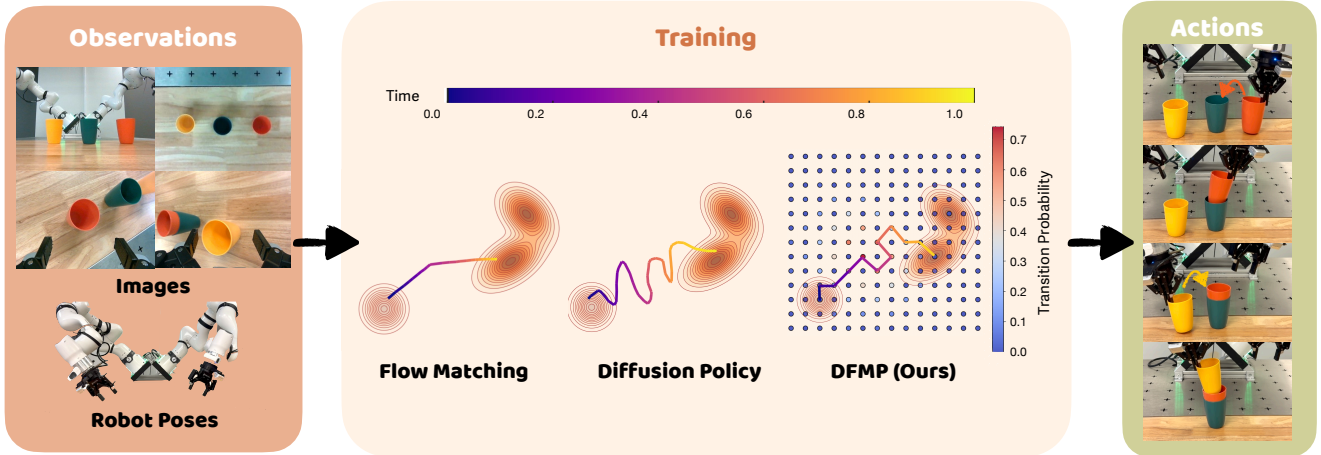


Fig. 1: Comparison of generative approaches for visuomotor policy learning. **Flow Matching** learns direct trajectories but is limited in capturing multimodality. **Diffusion Policy** effectively models complex action distributions but suffers from slow iterative inference. Our proposed **DFMP** represents continuous robot actions as discrete tokens and models their evolution with a continuous-time Markov chain, preserving distinct action modes while enabling efficient action generation.

Abstract—It is well established that discrete representations offer a natural framework for capturing multimodal structure and modeling transitions in continuous signals. Building on this idea, we introduce Discrete Flow Matching Policy (DFMP), a method for learning continuous robot actions in a discrete space. DFMP formulates action generation as a continuous-time Markov chain over action tokens, combining three desirable properties: (i) multimodal behavior modeling through probabilistic branching among tokens, (ii) fast inference through few-step sampling, and (iii) stable optimization through a flow-matching objective. To bridge discrete representations and continuous control, we systematically compare action tokenization schemes, analyze their trade-offs, and identify an effective scheme for real-world robot policies. Experiments across a broad range of simulated manipulation benchmarks and two real-world robot deployments demonstrate strong task performance and improved scalability and robustness relative to continuous-space baselines. These results support DFMP as a principled framework for efficient and robust visuomotor policy learning. Videos, code and additional results are available at <https://dfm-policy.github.io/>.

I. INTRODUCTION

Humans acquire dexterous motor skills through observation and practice, drawing on prior experience to adapt to new tasks and contexts [1]. This ability to discover patterns from diverse experience and reuse them in other tasks is what makes human intelligence so adaptable. Learning from diverse data has also driven progress in vision and language, where models trained on large-scale datasets exhibit broad generalization and transfer across domains [2], [3]. In robotics, scaling visuomotor policy learning with diverse demonstration data offers a promising path from task-specific behaviors toward

flexible, general-purpose control [4]–[6]. Realizing this potential requires policy models that can capture the diversity of behaviors present in these datasets.

Despite substantial progress in learning robotic policies from demonstrations [7], [8], preserving this behavioral diversity remains challenging. Demonstrations often contain multiple valid responses to the same observation, with differences that matter at fine spatial and temporal scales. For example, an object may be grasped from several distinct end-effector poses, and a manipulation task may admit multiple successful trajectories. Averaging across incompatible action modes can produce commands that correspond to none of these valid behaviors, leading to unsuccessful or unstable execution [9]. Effective policies must therefore preserve distinct action alternatives while maintaining the precision required for control.

Diffusion policies model multimodal action distributions through iterative denoising, although repeated sampling increases inference cost [10]. Flow matching offers stable and efficient training for continuous generative models [11]. Beyond task success, evaluation must consider whether these models preserve fine-grained action modes and remain robust to changes in object shape. We therefore ask: **How can we design efficient visuomotor policies that preserve fine-grained action multimodality and remain robust to geometric variation?**

Discrete tokens model action alternatives through probability mass assigned to distinct tokens and probabilistic transitions between them. We introduce Discrete Flow Matching

Policy (DFMP) for learning continuous robot actions in a discrete space. DFMP models token generation as a continuous-time Markov chain and learns its transition dynamics through discrete flow matching [12]. This formulation supports stable optimization, multimodal action modeling, and fast inference. Our contributions are (i) a discrete flow-matching framework for visuomotor policy learning; (ii) ablations that isolate the contribution of discrete flow matching from action tokenization; and (iii) comparisons with state-of-the-art generative visuomotor policies across eight simulated and two real-world experiments, evaluating task performance, multimodal behavior preservation, and robustness to geometric variations, including changes in object shape.

II. RELATED WORK

A. Learning from Demonstrations

Robots acquire skills by imitating demonstrations [13], [14], collected through kinesthetic teaching, teleoperation, or scripted controllers [15]. Behavior cloning (BC) maps observations to actions [16]–[18], but faces mode averaging under deterministic squared-error regression [19], poor generalization with limited data coverage [20], and compounding errors from covariate shift [21]–[23]. Extensions include Dataset Aggregation (DAgger) [21], DART noise injection [24], hierarchical policies [9], and large-scale imitation frameworks [7].

B. Generative Modeling for Policy Learning

Generative policies use energy-based models [25], conditional variational models [26], and diffusion models [27] adapted to visuomotor control [10] and trajectory planning [28]. Diffusion policies capture multimodal behavior [29] and achieve strong manipulation performance [30], although iterative denoising can make inference costly. Flow matching [11] and related stochastic-interpolant frameworks [31] learn continuous transport dynamics that can support efficient sampling. Vision-language-action models apply flow matching to generate continuous action chunks [32], [33]. DFMP learns stochastic transitions between discrete action tokens, with evaluations examining task performance and multimodal behavior preservation.

C. Action Tokenization and Discrete Generative Policies

Action tokenization provides an interface between continuous control and discrete prediction. RT-2 and OpenVLA discretize action dimensions for prediction with vision-language backbones [5], [34]. Behavior Transformer combines k -means action bins with continuous residual corrections [19], whereas VQ-BeT learns hierarchical vector-quantized action representations [35]. FAST compresses action sequences using a frequency-space representation [36]. More recently, Ordered Action Tokenization (OAT) learns ordered action tokens whose prefixes decode into progressively refined action chunks, enabling a trade-off between inference cost and reconstruction fidelity [37].

The choice of generative process is distinct from the choice of tokenizer. Discrete flow matching provides a framework

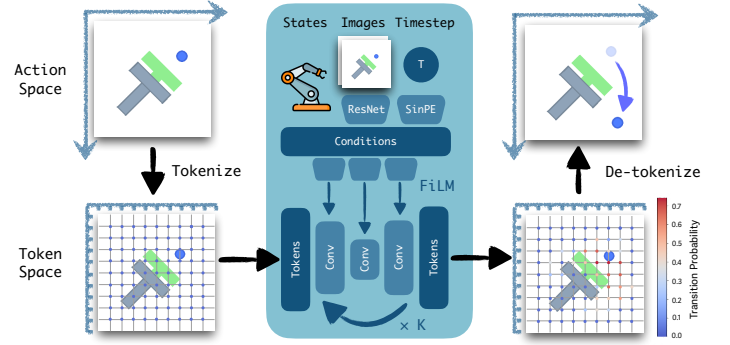


Fig. 2: Overview of DFMP. Continuous action chunks are tokenized, generated through observation-conditioned discrete flow matching, and decoded into executable actions.

for transporting probability mass through continuous-time Markov chains over discrete states [12]. Recent robotic policies increasingly adopt discrete generative processes: Discrete Diffusion VLA iteratively denoises action tokens with parallel decoding and re-masking [38], while Unified Diffusion VLA jointly denoises future visual tokens and action tokens [39]. Closely related to our formulation, DFM-VLA applies discrete flow matching to iterative action-token refinement within a vision-language-action model [40]. VQActFlow instead combines a learned action codebook with variational flow matching in the continuous space of codebook embeddings and inference-time mode guidance [41]. Together, these approaches illustrate that discrete action representations can support different generative dynamics. DFMP studies discrete-state action generation with particular emphasis on multimodal behavior preservation and ablations that isolate the contribution of discrete flow matching from action tokenization alone.

III. METHODS

We introduce **Discrete Flow Matching Policy (DFMP)**, which formulates visuomotor policy learning as conditional generative modeling over discrete action tokens. As illustrated in Fig. 2, DFMP encodes continuous actions into tokens, learns observation-conditioned transitions between tokens, and decodes generated tokens into executable actions.

A. Discrete Flow Matching Policy Formulation

1) *Continuous-Time Markov Chain*: We model action-token generation as a continuous-time Markov chain (CTMC) on a discrete token state space $\mathcal{X}$, conditioned on an observation context c . Let $\tau \in [0, 1]$ denote generative time; the observation context remains fixed throughout generation. Define the transition kernel as $p_{\tau+h|\tau}(y | x, c) = \mathbb{P}(X_{\tau+h} = y | X_{\tau} = x, c)$. For a small increment $h > 0$, it satisfies

$$p_{\tau+h|\tau}(y | x, c) = \delta(y, x) + h u_{\tau}(y, x | c) + o(h), \quad (1)$$

where $\delta(y, x)$ is the Kronecker delta and $u_{\tau}(y, x | c)$ is the transition rate from x to y . A valid rate field satisfies $u_{\tau}(y, x | c) \geq 0$ for $y \neq x$ and $\sum_{y \in \mathcal{X}} u_{\tau}(y, x | c) = 0$. Thus, off-diagonal entries specify rates of transition to other states, while the diagonal entry is the negative total outgoing rate.

2) *Discrete Probability Paths*: Flow matching constructs a probability path connecting a source distribution to a target distribution [11], [12]. We tokenize each continuous action chunk as a sequence in $\mathcal{X} = \mathcal{V}^N$, where N is the number of token positions and $\mathcal{V}$ is a vocabulary of size V . Let p_{src} denote the source-token distribution and $q(\cdot | c)$ the target action-token distribution conditioned on observations. For each context c , let $\pi_c(x_0, x_1)$ be a coupling with these source and target marginals. Averaging endpoint-conditioned paths gives

$$p_\tau(x | c) = \sum_{x_0, x_1 \in \mathcal{X}} p_\tau(x | x_0, x_1) \pi_c(x_0, x_1). \quad (2)$$

We construct the conditional path independently across token positions as $p_\tau(x | x_0, x_1) = \prod_{i=1}^N p_\tau(x^i | x_0^i, x_1^i)$, with boundary conditions $p_0(x^i | x_0^i, x_1^i) = \delta(x^i, x_0^i)$ and $p_1(x^i | x_0^i, x_1^i) = \delta(x^i, x_1^i)$. This conditional factorization does not require the marginal action-token distribution to factorize.

We use the quadratic schedule $\kappa(\tau) = \tau^2$, yielding

$$p_\tau(x^i | x_0^i, x_1^i) = (1 - \tau^2) \delta(x^i, x_0^i) + \tau^2 \delta(x^i, x_1^i). \quad (3)$$

The mixture weight shifts from the source token to the target token as τ increases, with zero initial slope because $\kappa'(0) = 0$.

3) *Discrete Velocity Field*: The transition-rate field, also called the discrete velocity field, generates the prescribed path when it satisfies the Kolmogorov forward equation:

$$\frac{d}{d\tau} p_\tau(y | c) = \sum_{x \in \mathcal{X}} u_\tau(y, x | c) p_\tau(x | c). \quad (4)$$

The right-hand side represents net probability flow into state y , including outflow through the diagonal term.

For the path in Eq. (3) and $0 \leq \tau < 1$, a valid transition-rate field at token position i is

$$u_\tau^i(y^i, x^i | x_0^i, x_1^i) = \frac{2\tau}{1 - \tau^2} [\delta(y^i, x_1^i) - \delta(y^i, x^i)]. \quad (5)$$

The full conditional generator is obtained by summing these coordinate-wise rates, with each transition changing one token position and leaving the others unchanged. Averaging the conditional generators over the endpoint posterior yields the marginal field:

$$u_\tau(y, x | c) = \mathbb{E}[u_\tau(y, x | X_0, X_1) | X_\tau = x, c]. \quad (6)$$

This relation provides the basis for learning the marginal dynamics from tractable conditional targets.

B. Policy Learning and Execution

1) *Training Objective*: We parameterize the observation-conditioned transition-rate field as u_τ^θ , where θ denotes the trainable network parameters. During training, we obtain observation contexts and target action tokens from demonstrations, draw source tokens according to the chosen coupling, sample a generative time τ , and draw an intermediate state $x_\tau \sim p_\tau(\cdot | x_0, x_1)$. For brevity, denote the conditional target vector by $\mathbf{u}_\tau^{\text{cond}} = u_\tau(\cdot, x_\tau | x_0, x_1)$ and the predicted vector by $\mathbf{u}_\tau^\theta = u_\tau^\theta(\cdot, x_\tau | c)$. We optimize

$$\mathcal{L}_{\text{CDFM}}(\theta) = \mathbb{E}_{c, (x_0, x_1), \tau, x_\tau} [D_{x_\tau}(\mathbf{u}_\tau^{\text{cond}}, \mathbf{u}_\tau^\theta)], \quad (7)$$

where $(x_0, x_1) \sim \pi_c$ and D_{x_τ} is a Bregman divergence defined on valid transition-rate vectors at state x_τ . The first argument is the conditional target, and the second is the learned rate field. Under the conditions established by [42], this objective has the same expected parameter gradients as the corresponding marginal discrete flow-matching objective, which uses the field in Eq. (6) as its target. Training can therefore use conditional targets without explicitly computing the endpoint posterior.

2) *Network Architecture*: We adopt a CNN-based U-Net, building on architectures used for generative visuomotor policies [10], [43]. The network takes the intermediate token sequence x_τ , generative time τ , and observation context c as inputs and produces vocabulary logits at each token position. A softmax over the vocabulary dimension independently normalizes each position's output. These token probabilities are used to parameterize the learned transition dynamics u_τ^θ . For visual observations, we use ResNet-18 [44] to extract image features, which are combined with proprioceptive information to condition the policy. The network processes the action-token sequence jointly, allowing each token prediction to depend on the surrounding action context.

3) *Receding-Horizon Execution*: At control step k , the observation context c_k comprises the most recent T_o visual and proprioceptive observations. Conditioned on this context, DFMP initializes a token sequence from the source distribution and generates action tokens using the learned CTMC dynamics. The decoder maps these tokens to an action chunk $A_k = (a_k, \dots, a_{k+T_p-1}) \in \mathbb{R}^{T_p \times d_a}$, where T_p is the prediction horizon and d_a is the dimension of each continuous action. The robot executes the first T_a actions, then replans using updated observations. This receding-horizon procedure incorporates feedback while retaining a longer prediction horizon.

IV. EXPERIMENTS

We systematically evaluate DFMP on eight simulated and two real-world experiments, and perform rigorous experiments and ablations. We aim to answer the following questions: **RQ1**: How does DFMP compare against state-of-the-art generative visuomotor policy learning approaches across simulated and real-world tasks? **RQ2**: How well does DFMP preserve multimodal behaviors compared with continuous generative policy baselines? **RQ3**: How robust is DFMP to geometric variations, including changes in the shape of the manipulated object? **RQ4**: Are the performance gains of DFMP attributable to the discrete flow-matching formulation itself, rather than merely to action tokenization?

A. Baselines

We compare against two generative policy baselines. **Flow Matching (FM)** uses the same U-Net backbone with the continuous flow-matching objective of [11]. **Diffusion Policy (DP)** uses the DDPM objective following [10]. We consider DP with 10 generation steps to match the other methods and with 100 steps to follow the original paper's configuration.

For **DFMP**, we compare two complementary tokenization strategies: uniform per-dimension discretization and data-adaptive joint-space partitioning. Both tokenizers are fitted

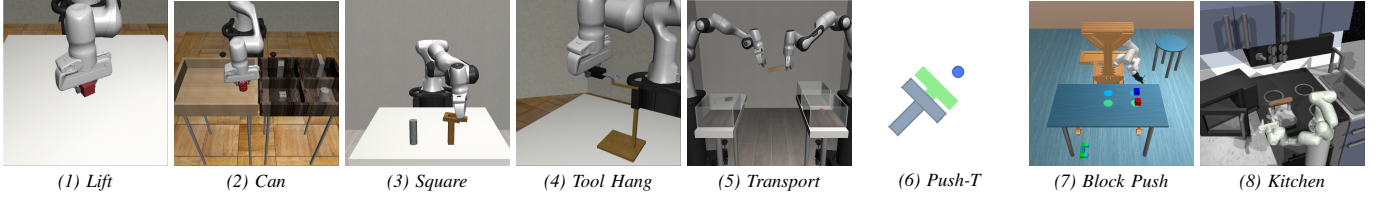


Fig. 3: **Simulated Environments.** *Robomimic Benchmark* [45] (1-5), *Push-T* [46] (6), *UR3 Block Push* [19] (7) and *Franka Kitchen* [47](8).

TABLE I: Simulation Benchmark Success Rates

Multimodality	Task	HiPrec ^a	FM (10 steps)	DP (10 steps)	DP (100 steps)	DFMP-B (10 steps)	DFMP-K (10 steps)
Low	Lift		1.00/0.98^b	1.00/0.95	1.00/0.96	1.00/0.99	1.00/0.97
	Can		1.00/0.98	0.98/0.92	0.98/0.94	0.98/0.95	0.92/0.88
	Square	✓	0.88/0.77	0.84/0.75	0.90/0.78	0.92/0.84	0.88/0.79
	Tool Hang	✓	0.68/0.58	0.56/0.40	0.70/0.61	0.68/0.57	0.76/0.68
	Transport	✓	0.92/0.79	0.86/0.79	0.94/0.87	0.96/0.86	0.92/ 0.88
High	Kitchen		0.88/0.84	0.98/0.94	1.00/0.97	1.00/0.97	0.98/0.95
	Block Push		0.96/0.80	0.88/0.76	0.94/0.79	0.98/0.84	0.92/0.79
	Push-T	✓	0.92/0.84	0.94/0.82	0.98/0.88	1.00/0.86	0.98/ 0.94
Mean			0.91/0.82	0.88/0.79	0.93/0.85	0.94/0.86	0.92/ 0.86

^a HiPrec denotes high-precision tasks.

^b Entries report the best-checkpoint success rate followed by the mean success rate over the final three evaluation checkpoints, both averaged across three training seeds. Each checkpoint is evaluated over 50 rollouts per seed.

once before policy training without gradient-based optimization and kept fixed thereafter. The **bin tokenizer** (DFMP-B) divides each action dimension into B uniform bins of width $\Delta^d = (u^d - l^d)/B$, decoding each token to its bin center. This simple representation tests whether DFMP is effective without sophisticated tokenization. The **KD-tree tokenizer** (DFMP-K) recursively partitions the joint action space through axis-aligned median splits into V leaves, encoding each action as $k = \text{leaf}(a)$. Its data-adaptive partitions account for the joint action distribution, allowing us to assess the additional benefit of exploiting cross-dimensional structure. We keep the policy architecture and training procedure fixed across variants.

B. Simulation Benchmarks

We evaluate DFMP on eight simulated tasks across four benchmark suites, covering both state- and vision-based observations and a range of control precision and task horizons. *Robomimic* [45] includes five manipulation tasks: *Lift*, *Can*, *Square*, *Tool Hang*, and *Transport*. *Push-T* [46] requires pushing a T-shaped block from randomized initial configurations to a target region. *UR3 Block Push* [19] requires pushing two blocks to their respective goal regions. *Franka Kitchen* [47] is a multi-stage manipulation benchmark with a 7-DoF Franka arm; we report the *p1* success rate, i.e., the fraction of episodes in which at least one target interaction is successfully completed. The tasks are illustrated in Fig. 3.

1) *Main Results:* Table I demonstrates that DFMP combines accurate manipulation with strong performance on multimodal tasks using only 10 generation steps. FM is already

highly effective on simpler tasks such as Lift and Can, but these results do not establish its ability to preserve diverse behaviors. Our Push-T analysis (Sec. IV-B2) exposes this distinction: from a shared initial state, the evaluated FM policy concentrates successful rollouts on a single route, whereas both DFMP variants retain multiple successful routes. Meanwhile, DFMP’s competitive or stronger results on Square, Tool Hang, and Transport show that discrete action generation remains compatible with precision-sensitive control. DP achieves strong performance through iterative denoising, but its aggregate mean performance reaches 0.85 with 100 steps, compared with 0.86 for both DFMP variants with 10 steps. DFMP therefore combines behavioral diversity and precise execution with substantially fewer generation steps than the evaluated 100-step DP baseline.

Figure 4 shows that both FM and DFMP-K improve rapidly early in training, while DP takes longer to reach competitive performance. DFMP-K combines this fast initial progress with stronger late-training results on both tasks. Additional denoising steps increase DP’s rollout evaluation cost, contributing to longer elapsed times when evaluation is included. However, DFMP-K also improves earlier than DP at the same generation-step setting, so its observed time advantage extends beyond the difference in sampling cost.

The two tokenizers allocate action resolution differently: uniform binning discretizes each coordinate independently, whereas KD-tree tokenization adapts joint-space partitions to the demonstrated action distribution. Their comparable

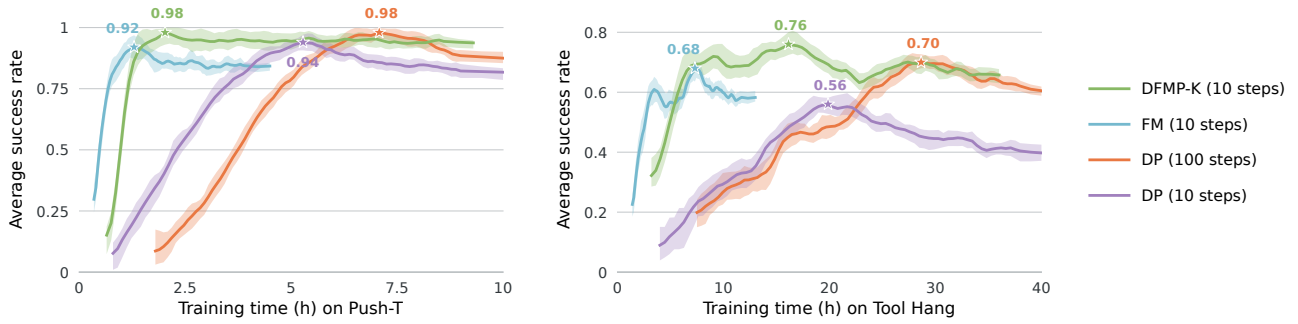


Fig. 4: DFMP achieves faster convergence and higher success across *Push-T* and *Tool Hang*.

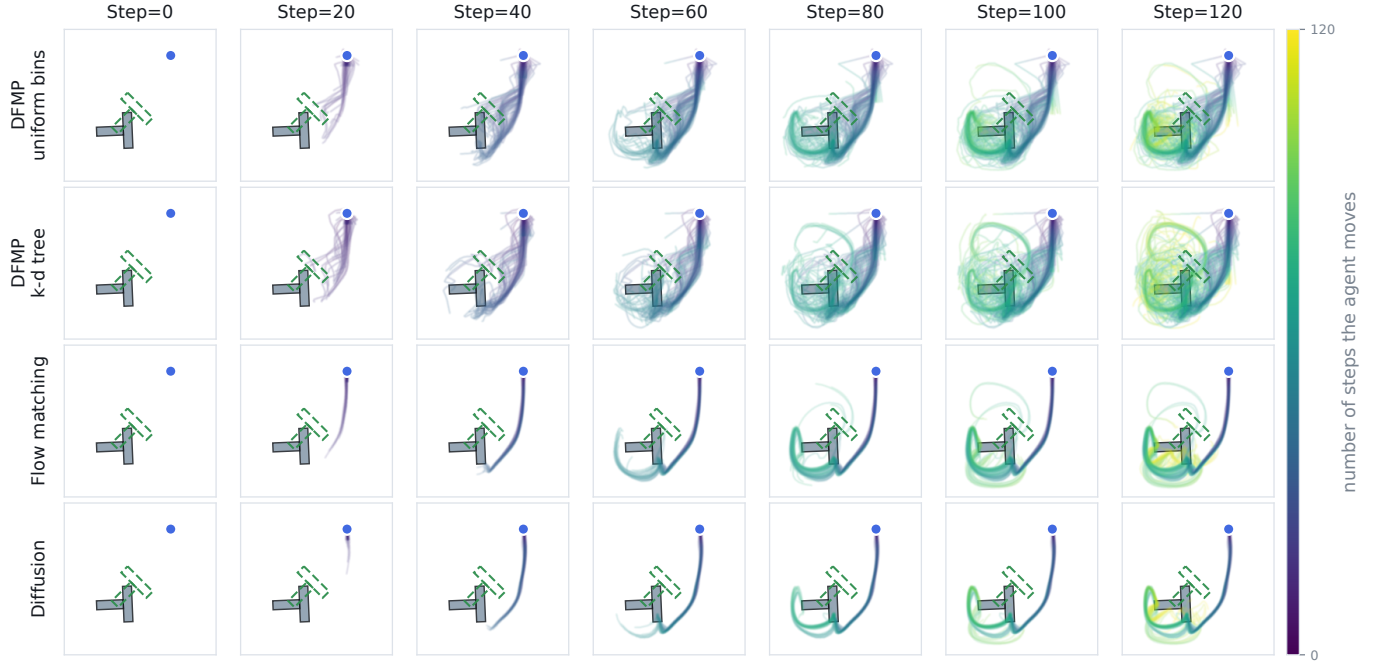


Fig. 5: Push-T rollout distributions for 200 successful rollouts per policy from a shared initial state. Color and opacity encode mean visitation step and frequency; green marks the goal, the dashed outline the initial block pose, and blue the agent start. DFMP shows broader trajectory distributions than Flow Matching and Diffusion Policy, suggesting greater behavioral diversity.

aggregate performance shows that data-adaptive partitioning is not required for strong overall results. Its benefits are task-specific, with KD-tree tokenization performing better on Tool Hang and Push-T, both of which require coordinated action components during contact-rich manipulation.

2) *Multimodality*: DFMP shows multimodal behavior on Push-T. As shown in Fig. 5, DFMP generates several successful trajectory patterns from the same initial state, while FM and DP follow more deterministic paths. Table II supports this observation: DFMP-B discovers two distinct routes, and DFMP-K discovers all four pooled routes, with higher trajectory-cluster entropy and more effective modes than the baselines. DFMP also has larger mean spread and entropy in its rollouts than FM and DP. These results indicate that DFMP produces more diverse behaviors, although its route distribution remains imbalanced.

TABLE II: Policy Multimodality on Push-T

Policy	Split (%) ^a	No. Routes	Traj. Entropy ^b	Eff. Modes	Traj. Spread ^c
DFMP-B	88.5/11.5	2	0.327	1.25	68.3
DFMP-K	95.5/4.5	4	0.492	1.41	87.5
FM	100.0/0.0	1	0.000	1.00	51.1
DP	100.0/0.0	1	0.000	1.00	37.6

^a Major/minor mode shares of successful rollouts, based on the pre-contact signed angle around the block; 100/0 indicates a single mode.

^b Trajectory-cluster entropy in bits.

^c Average pairwise trajectory distance at step 120.

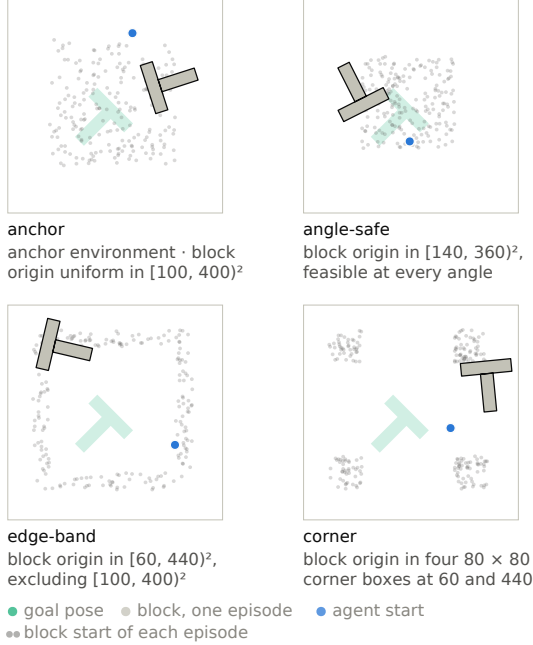
3) *Out-of-Distribution Robustness*: To evaluate the zero-shot robustness of DFMP, we use Push-T as a proxy task. Each policy is trained on the anchor setting and evaluated without adaptation under the spatial and geometric shifts shown in

TABLE III: Push-T Out-of-Distribution Tasks

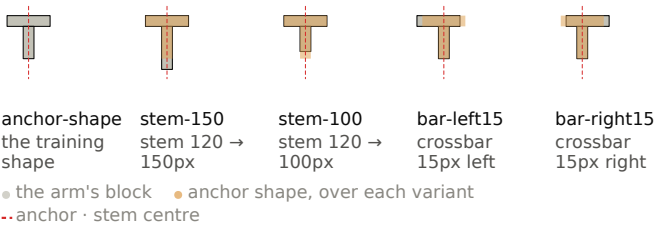
Policy	ID	Spatial OOD Settings				Geometric OOD Settings			
		Anchor	Angle-Safe	Edge-Band	Corner	Stem-150	Stem-100	Bar-Left	Bar-Right
DFMP-B	0.98 / 0.87 ^{a,b}	0.92 / 0.82	0.88 / 0.92	0.74 / 0.65	0.74 / 0.65	0.68 / 0.45	0.16 / 0.28	0.52 / 0.46	0.60 / 0.56
DFMP-K	1.00 / 0.94	0.98 / 0.96	0.94 / 0.97	0.84 / 0.68	0.78 / 0.80	0.78 / 0.69	0.36 / 0.39	0.54 / 0.33	0.62 / 0.54
FM	1.00 / 0.90	0.96 / 0.89	0.94 / 0.93	0.70 / 0.71	0.88 / 0.75	0.88 / 0.77	0.32 / 0.23	0.60 / 0.63	0.54 / 0.45
DP	0.96 / 0.79	0.88 / 0.87	0.92 / 0.86	0.78 / 0.75	0.86 / 0.69	0.82 / 0.75	0.08 / 0.11	0.42 / 0.43	0.38 / 0.43

^a Entries report the best-checkpoint success rate followed by the mean success rate over the final three evaluation checkpoints, both averaged across three training seeds. Each checkpoint is evaluated over 50 rollouts per seed.

^b All policies are evaluated zero-shot.



(a) Spatial out-of-distribution settings.



(b) Geometric out-of-distribution settings.

Fig. 6: Push-T out-of-distribution evaluation settings: (a) initial block distributions for the anchor and spatial variant settings; (b) block-shape variations for geometric variant settings.

Fig. 6. The spatial tests vary the block orientation and initial position, while the geometric tests modify the block shape. As shown in Table III, DFMP-K achieves the best or tied-best result in most spatial settings, including Anchor, Angle-Safe, Edge-Band, and Corner. It also performs best on Stem-100 and achieves the highest best-checkpoint success rate on Bar-Right. Although DFMP does not outperform the baselines on every

TABLE IV: Ablation of Action Tokenization and Flow Objectives on Push-T

	Method	Tokenizer	Discrete Action Space	Discrete Flow Objective	Success Rate ^a
(i)	DFMP	Bin	✓	✓	1.00/0.86
(i)	DFMP	KD-Tree	✓	✓	0.98/0.94
(ii)	CFM	Bin	✓	×	0.90/0.83
(ii)	CFM	KD-Tree	✓	×	0.90/0.88
(iii)	CFM	—	×	×	0.92/0.84

^a Entries report the best-checkpoint success rate followed by the mean success rate over the final three evaluation checkpoints, both averaged across three training seeds. Each checkpoint is evaluated over 50 rollouts per seed.

shape variant, its strong performance across both types of shift demonstrates robust generalization to unseen task settings.

4) *Contribution of the Discrete Flow Formulation:* To distinguish the contribution of discrete flow matching from action tokenization, we compare (i) DFMP on action tokens, (ii) CFM on tokenized actions, and (iii) CFM on raw actions. All configurations use the same policy backbone, training settings, and three training seeds, with 10 generation steps. (i) and (ii) share the same fitted tokenizer. Comparing (ii) with (iii) evaluates the effect of tokenization while retaining the continuous flow formulation. Comparing (i) with (ii) evaluates the contribution of the discrete flow formulation with tokenization held fixed.

Table IV shows that tokenization alone does not consistently improve CFM: both tokenizers reduce best-checkpoint performance, while uniform binning lowers the final-three-checkpoint mean and KD-tree tokenization raises it. With the tokenizer held fixed, DFMP improves both metrics over tokenized-action CFM under both tokenizers. Both DFMP variants also outperform raw-action CFM on both metrics, demonstrating a net improvement over the continuous baseline. Together, these comparisons show that tokenization alone cannot explain the observed gains and support the additional contribution of the discrete flow formulation.

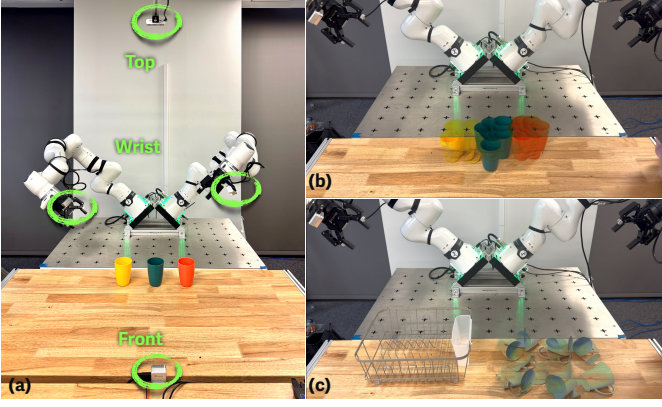


Fig. 7: Real-world setup. (a) Bimanual platform and camera placement. (b) Initial cup positions for *Cup Stacking*. (c) Initial cup poses and the target rack for *Cup Placing*.

TABLE V: Real-World Success Rates (across 50 runs)

Task	Setting	FM (10 steps)	DP (10 steps)	DP (100 steps)	DFMP-B (10 steps)
Cup Stacking	ID	0.84	0.80	0.90	0.90
Cup Stacking	OOD	0.76	0.74	0.84	0.86
Cup Placing	ID	0.76	0.84	0.90	0.92
Cup Placing	OOD	0.70	0.80	0.86	0.88

C. Real-World Tasks

1) *Experimental Setup*: We evaluate DFMP on two complementary real-world tasks using a bimanual Franka system with top, front, and two wrist cameras (Figure 7). *Cup Stacking* requires the two arms to sequentially stack three cups. Its main challenges are coordinating the arms and precisely aligning the cups to form a stable stack. *Cup Placing* requires the left arm to pick up a cup and hand it over to the right arm, which then places it into a dish rack. The task admits multiple valid grasping strategies and requires coordinated grasping, a stable handover, and accurate placement.

For each task, we collect 120 demonstrations at 30 Hz and execute robot control at 30 Hz. All methods are trained on a single NVIDIA H200 GPU and evaluated after completing their respective training schedules. We consider two evaluation settings: *in-distribution* (ID), with initial cup poses drawn from the training distribution, and *out-of-distribution* (OOD), which we use here to denote held-out initial cup poses not seen in the training demonstrations.

2) *Results*: Table V shows that DFMP-B matches or exceeds all baselines across the four task–setting combinations while using 10 generation steps. The overall trends are consistent with the simulation results: DFMP performs strongly on a task with multiple valid grasping strategies without an observed success-rate penalty on precision-sensitive task.

a) *Precision-sensitive manipulation*: On *Cup Stacking*, DFMP-B matches the strongest continuous baseline under ID evaluation. Since successful stacking requires coordinated arm

motion and precise cup alignment, this result suggests that the tokenized action representation retains sufficient resolution for the tested manipulation task. Discretization therefore need not preclude accurate execution, consistent with the simulation findings.

b) *Performance on a multimodal task*: On *Cup Placing*, DFMP-B performs best under ID evaluation while exhibiting grasps at the cup wall, handle, base, and side, with no observed mode collapse. Effective behavior on this task requires both retaining alternative grasping strategies and executing a selected strategy coherently. The observed grasp diversity, together with strong task performance, suggests that DFMP-B supports both aspects of multimodal manipulation.

c) *Generalization to unseen initial poses*: DFMP-B maintains strong performance on held-out initial cup poses, suggesting that its learned behavior can accommodate changes in the spatial configuration of the task. Across both tasks, we also observe recovery behavior: DFMP-B reattempts grasping after a missed grasp. Such retries provide qualitative evidence of closed-loop adaptation to execution errors, complementing the evaluation of generalization to unseen initial conditions.

V. CONCLUSION AND LIMITATIONS

We introduced DFMP, which generates continuous robot actions through discrete flow matching over action tokens. Across eight simulated and two real-world tasks, DFMP combines competitive task performance with few-step generation, while supporting behavioral diversity and precision-sensitive execution. Controlled Push-T ablations show that tokenization alone cannot explain the gains, supporting the contribution of the discrete flow formulation. Evaluations under spatial and geometric variations further demonstrate generalization within the tested settings. Together, these findings support discrete flow matching as an effective approach to visuomotor policy learning.

Several limitations remain. Tokenizers are fitted independently and fixed during policy training, leaving action resolution dependent on the chosen partition and vocabulary size. Although DFMP retains multiple behavioral modes, their sampling frequencies remain imbalanced. Real-world evidence is limited to two tasks and initial-pose variation, with grasp diversity and recovery assessed qualitatively. Future work should investigate task-aware tokenization, quantify mode coverage and recovery reliability, and evaluate scalability with larger models and more diverse data.

ACKNOWLEDGMENTS

We acknowledge the use of OpenAI’s ChatGPT to assist with language editing throughout the manuscript, searching for relevant literature, and debugging experimental code. The authors take full responsibility for the final manuscript, implementation, and results.

REFERENCES

- [1] B. M. Lake, T. D. Ullman, J. B. Tenenbaum, and S. J. Gershman, "Building machines that learn and think like people," *Behavioral and Brain Sciences*, 2017.
- [2] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal *et al.*, "Learning transferable visual models from natural language supervision," in *ICML*, 2021.
- [3] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal *et al.*, "Language models are few-shot learners," *NeurIPS*, 2020.
- [4] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, and *et al.*, "Rt-1: Robotics transformer for real-world control at scale," *RSS*, 2022.
- [5] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski *et al.*, "Rt-2: Vision-language-action models transfer web knowledge to robotic control," *arXiv*, 2023.
- [6] A. Brohan, S. Chen, D. Driess, F. Xia, and *et al.*, "Rt-x: Scaling up robot learning with diverse datasets," *arXiv*, 2023.
- [7] E. Jang, A. Irpan, M. Khansari, D. Kappler, F. Ebert, C. Lynch *et al.*, "Bc-z: Zero-shot task generalization with robotic imitation learning," *arXiv*, 2022.
- [8] A. Zeng, P. Florence, J. Tompson, S. Welker, J. Chien, M. Attarian *et al.*, "Transporter networks: Rearranging the visual world for robotic manipulation," in *CoRL*, 2021.
- [9] C. Lynch, M. Khansari, T. Xiao, V. Kumar, J. Tompson, S. Levine *et al.*, "Learning latent plans from play," *arXiv*, 2019.
- [10] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel *et al.*, "Diffusion policy: Visuomotor policy learning via action diffusion," in *RSS*, 2023.
- [11] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," *arXiv*, 2023.
- [12] I. Gat, T. Remez, N. Shaul, F. Kreuk, R. T. Q. Chen, G. Synnaeve *et al.*, "Discrete flow matching," *arXiv*, 2024.
- [13] S. Schaal, "Is imitation learning the route to humanoid robots?" *Trends in cognitive sciences*, 1999.
- [14] A. J. Ijspeert, J. Nakanishi, and S. Schaal, "Movement imitation with nonlinear dynamical systems in humanoid robots," in *ICRA*. IEEE, 2002.
- [15] B. D. Argall, S. Chernova, M. Veloso, and B. Browning, "A survey of robot learning from demonstration," *Robotics and autonomous systems*, 2009.
- [16] M. Bain and C. Sammut, "A framework for behavioural cloning," in *Machine Intelligence*, 1995.
- [17] D. A. Pomerleau, "ALVINN: An autonomous land vehicle in a neural network," in *NIPS*, 1989.
- [18] F. Torabi, G. Warnell, and P. Stone, "Behavioral cloning from observation," in *IJCAI*, 2018.
- [19] N. M. M. Shafiuallah, Z. J. Cui, A. Altanzaya, and L. Pinto, "Behavior transformers: Cloning k modes with one stone," *arXiv*, 2022.
- [20] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni *et al.*, "What matters in learning from offline human demonstrations for robot manipulation," *arXiv*, 2021.
- [21] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," *arXiv*, 2011.
- [22] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, "Hg-dagger: Interactive imitation learning with human experts," *arXiv*, 2019.
- [23] T. Osa, J. Pajarinen, G. Neumann, J. A. Bagnell, P. Abbeel, and J. Peters, "An algorithmic perspective on imitation learning," *Foundations and Trends® in Robotics*, 2018.
- [24] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg, "Dart: Noise injection for robust imitation learning," *arXiv*, 2017.
- [25] P. Florence, C. Lynch, A. Zeng, O. Ramirez, A. Wahid, L. Downs *et al.*, "Implicit behavioral cloning," in *CoRL*, 2022.
- [26] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," *arXiv*, 2023.
- [27] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *arXiv*, 2020.
- [28] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine, "Planning with diffusion for flexible behavior synthesis," *arXiv*, 2022.
- [29] T. Pearce, T. Rashid, A. Kanervisto, D. Bignell, M. Sun, R. Georgescu *et al.*, "Imitating human behaviour with diffusion models," *arXiv*, 2023.
- [30] T. Z. Zhao, J. Tompson, D. Driess, P. Florence, K. Ghasemipour, C. Finn *et al.*, "Aloha unleashed: A simple recipe for robot dexterity," *arXiv*, 2024.
- [31] M. S. Albergo, N. M. Boffi, and E. Vanden-Eijnden, "Stochastic interpolants: A unifying framework for flows and diffusions," *arXiv*, 2025.
- [32] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn *et al.*, " π_0 : A vision-language-action flow model for general robot control," *arXiv*, 2024.
- [33] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess *et al.*, " $\pi_{0.5}$: a vision-language-action model with open-world generalization," *arXiv*, 2025.
- [34] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair *et al.*, "Openvla: An open-source vision-language-action model," *arXiv*, 2024.
- [35] S. Lee, Y. Wang, H. Etukuru, H. J. Kim, N. M. M. Shafiuallah, and L. Pinto, "Behavior generation with latent actions," in *Proceedings of the 41st International Conference on Machine Learning*, ser. PMLR, 2024.
- [36] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong *et al.*, "FAST: Efficient action tokenization for vision-language-action models," *arXiv:2501.09747*, 2025.
- [37] C. Liu, Y. Zhao, H. Chen, X. Han, J. Gao, E. Adeli *et al.*, "Ordered action tokens for visuomotor policy learning," *arXiv:2607.21670*, 2026.
- [38] Z. Liang, Y. Li, T. Yang, C. Wu, S. Mao, L. Pei *et al.*, "Discrete diffusion VLA: Bringing discrete diffusion to action decoding in vision-language-action policies," *arXiv*, 2025.
- [39] J. Chen, W. Song, P. Ding, Z. Zhou, H. Zhao, F. Tang *et al.*, "Unified diffusion VLA: Vision-language-action model via joint discrete denoising diffusion process," *arXiv*, 2025.
- [40] J. Chen, W. Song, J. Fang, R. Yin, J. Wang, S. Chen *et al.*, "DFM-VLA: Iterative action refinement for robot manipulation via discrete flow matching," *arXiv*, 2026.
- [41] Z. Zhao, M. Leggiero, Y. Chen, H. Liu, Y. Wu, H. Xue *et al.*, "VQActFlow: Vector-quantized action mode steering for multi-task robot manipulation," *arXiv*, 2026.
- [42] Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer *et al.*, "Flow matching guide and code," *arXiv*, 2024.
- [43] K. Rana, R. Lee, D. Pershouse, and N. Suenderhauf, "Imle policy: Fast and sample efficient visuomotor policy learning via implicit maximum likelihood estimation," in *RSS*, 2025.
- [44] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *CVPR*, 2016.
- [45] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni *et al.*, "What matters in learning from offline human demonstrations for robot manipulation," *arXiv*, 2021.
- [46] P. Florence, C. Lynch, A. Zeng, O. Ramirez, A. Wahid, L. Downs *et al.*, "Implicit behavioral cloning," *arXiv*, 2021.
- [47] A. Gupta, V. Kumar, C. Lynch, S. Levine, and K. Hausman, "Relay policy learning: Solving long-horizon tasks via imitation and reinforcement learning," *arXiv*, 2019.